



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 04 — Sincronización

Bloque 01

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



Lo que veremos en este bloque



Condición de carrera

Dos hilos que se pisan la misma variable



Lock

Exclusión mutua: uno a la vez



RLock

El cerrojo reentrante

Los hilos de la unidad pasada **comparten memoria**. Hoy aprendemos a turnarnos para no pisarnos.

Condición de carrera

La idea en una frase

“

Una **condición de carrera** ocurre cuando dos o más hilos acceden a la misma variable compartida a la vez y el resultado depende de qué hilo llegue primero: los incrementos se pierden y el resultado es incorrecto.



Es **el bug más clásico** de la programación concurrente: el programa *funciona casi siempre* y falla de forma intermitente, imposible de reproducir.



Por eso la sincronización se diseña **desde el principio**, no al final.

El experimento: 4 hilos, un contador

● ● ● carrera.py

Python

```
import threading, sys
contador = 0
sys.setswitchinterval(1e-6) # ⚠️ cortes de hilo muy frecuentes
def sumar(c):
    return c + 1
def incrementar():
    global contador
    for _ in range(200_000):
        contador = sumar(contador) # leer-sumar-escribir vía función
hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"Esperado: 800.000 | Obtenido: {contador}")
```

```
$ python carrera.py
```

```
Esperado: 800.000 | Obtenido: 346.564
```

```
Esperado: 800.000 | Obtenido: 375.671 ← ¡nunca 800.000!
```

¿Por qué falla? El += no es atómico

`contador += 1` parece una línea, pero por dentro hace **3 operaciones**:

1 Leer `contador` de memoria

2 Sumar 1

3 Escribir el resultado

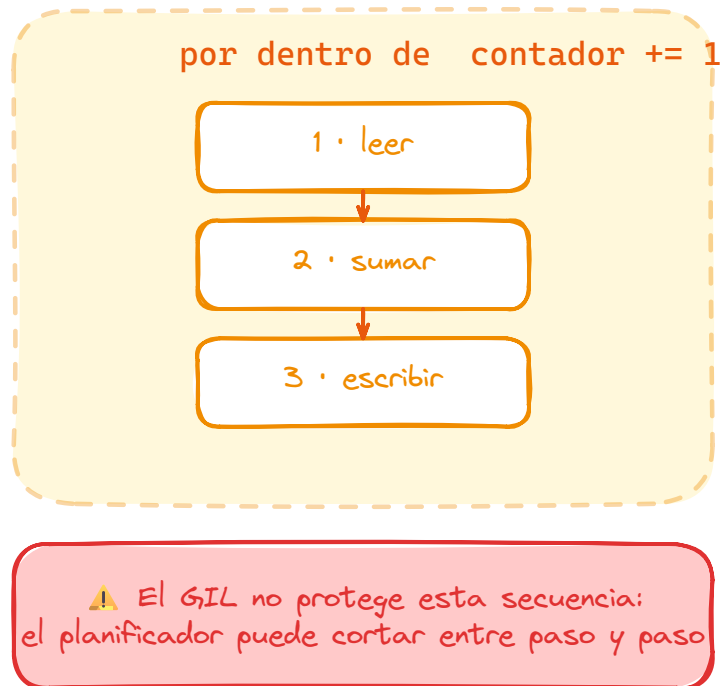
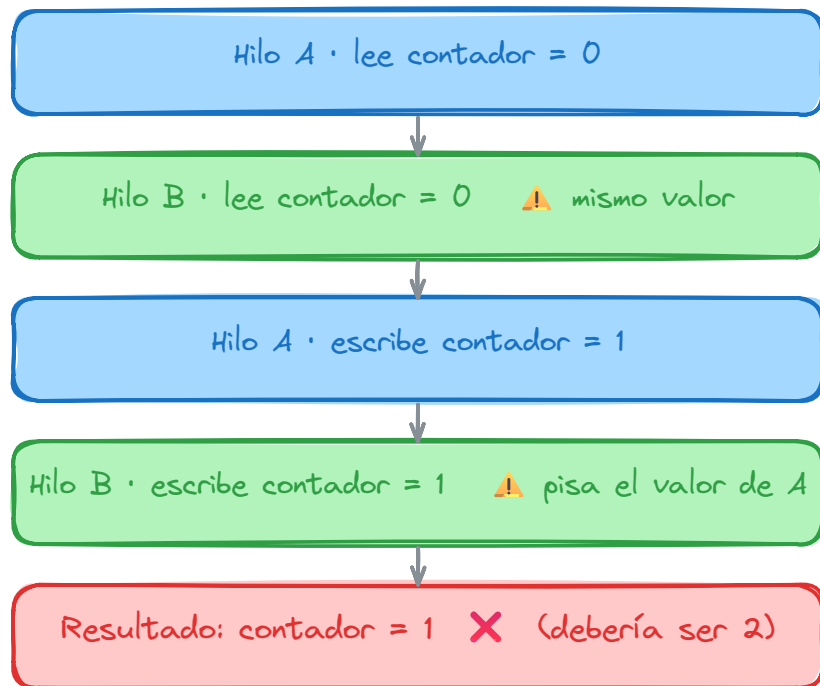
Si dos hilos leen el mismo valor antes de que ninguno escriba, **ambos escriben lo mismo** y se pierde un incremento.

⚠ En Python, el **GIL** evita que dos hilos ejecuten Python puro a la vez, pero **no** protege esta secuencia: el planificador puede interrumpir a un hilo entre el *leer* y el *escribir*.

i `sys.setswitchinterval(1e-6)` fuerza cortes muy frecuentes para **hacer visible** la carrera. La condición de carrera es real: solo hay que saber exponerla.

La traza del desastre

Condición de carrera — dos hilos, una variable compartida



La analogía: dos cajeros y una caja

- 1 El cajero A mira la caja: hay **100 €** (leer)
- 2 Mientras A busca cambio, B también mira: **100 €** (leer)
- 3 A mete 50 €: escribe **150 €**
- 4 B mete 50 €: también escribe **150 €**... ¡pisando el ingreso de A!

⚠ Se cobraron **200 €**, pero en la caja solo hay **150 €**: los 50 € de A **desaparecieron**.
Exactamente lo mismo le pasa a `contador`.

✓ El remedio es la **exclusión mutua**:
mientras un cajero toca la caja, el otro espera fuera. Ese es el **Lock**, el siguiente punto.

Dónde aparece la condición de carrera

Situación	Qué se pierde
Contador compartido (<code>contador += 1</code>)	Incrementos
Saldo de una cuenta bancaria	Ingresos / retiros
Cola de trabajos compartida	Elementos de la cola
Registro (log) con fecha y hora	Líneas de log
Caché con varios lectores/escritores	Datos actualizados



El programa **funciona casi siempre** y falla de forma intermitente e imposible de reproducir. Los tests no lo cazan: hay que **prevenirlo por diseño**.

Mini-chequeo: condición de carrera



Responde en 30 segundos:

1. ¿Qué es una condición de carrera?
2. ¿Qué tres operaciones esconde un `contador += 1` ?
3. ¿Cómo lo explicarías con la analogía de los dos cajeros?



Respuestas:

- Dos hilos tocan la misma variable a la vez y el resultado depende de quién llega primero
- **Leer** → **sumar** → **escribir**: si dos leen lo mismo, un incremento se pierde
- Dos cajeros leen 100 €, los dos escriben 150 € y el segundo ingreso desaparece

Lock

La idea en una frase

“

Un **Lock** (cerrojo) garantiza que solo un hilo entre en la **sección crítica** a la vez: el resto espera fuera hasta que el primero lo libera.



En el contador compartido, la sección crítica es `contador += 1`. Protegida con un Lock, los 4 hilos se **turnan** y el resultado vuelve a ser el esperado.



Con Lock, el contador pasa de resultados aleatorios a un **400.000 exacto** en cualquier ejecución.

Lock: métodos y código

con_lock.py

Python

```
import threading
contador = 0
lock = threading.Lock()
def incrementar():
    global contador
    for _ in range(100_000):
        with lock: # 🍷 solo un hilo aquí dentro
            contador += 1
hilos = [threading.Thread(target=incrementar)
         for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"Con Lock: {contador}") # ✅ 400.000
```

Método

Qué hace

`lock.acquire()`

Bloquea. Si otro hilo lo tiene, espera

`lock.release()`

Libera. Otro hilo puede entrar

`lock.locked()`

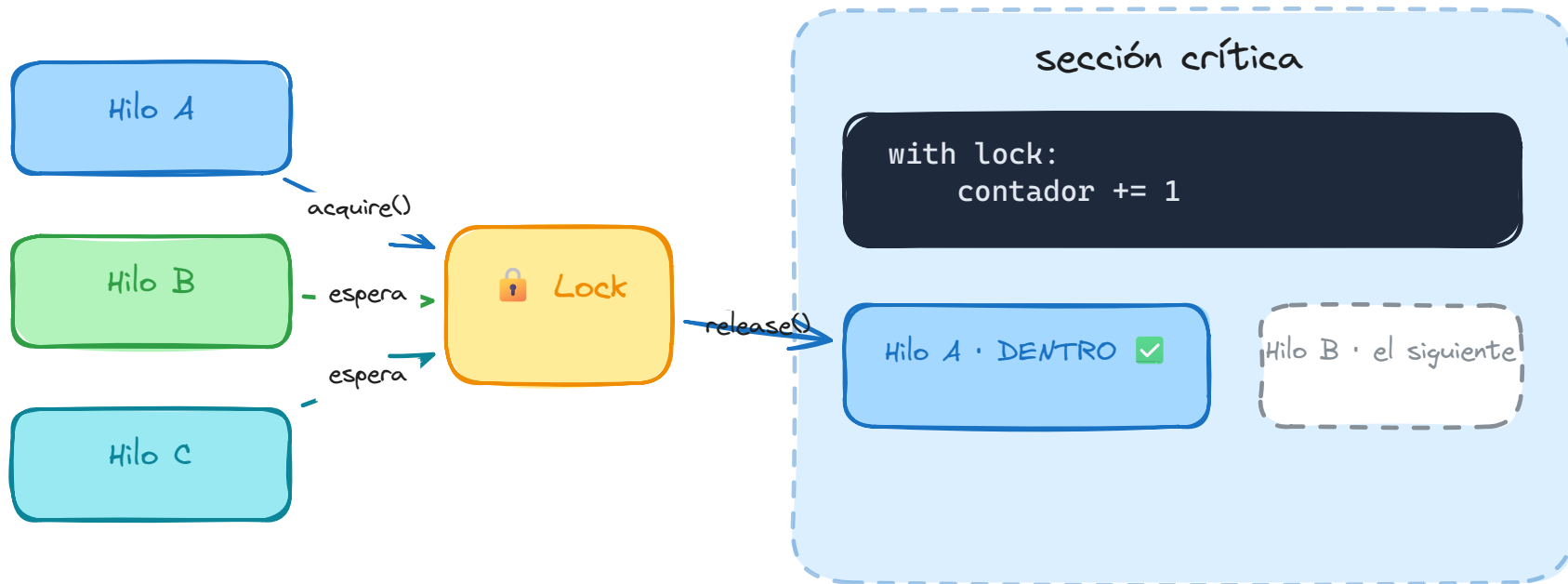
True si está bloqueado



El lock se adquiere **antes** de tocar el contador y se libera al salir del bloque `with`.

El cerrojo en acción

Lock — exclusión mutua sobre la sección crítica



💡 Regla de oro: todo el código que toca la variable compartida va dentro del mismo lock

Be the code: sin Lock vs con Lock

✗ Sin Lock — el caos

```
Hilo-A: lee contador = 0
```

```
Hilo-B: lee contador = 0 ← ¡mismo  
valor!
```

```
Hilo-A: escribe contador = 1
```

```
Hilo-B: escribe contador = 1 ← ¡pisó el  
incremento!
```

✓ Con Lock — el orden

```
Hilo-A: acquire() → lee 0 → escribe 1 →  
release()
```

```
Hilo-B: acquire() → lee 1 → escribe 2 →  
release() ← esperó su turno
```

i Con el Lock, el *leer* → *sumar* → *escribir* de cada hilo ocurre **de principio a fin sin que nadie se cuele**. Los cajeros ya no se pisan.

Siempre with lock:

✓ La forma segura

```
with lock:  
    contador += 1
```

El `release()` es automático, **incluso si hay una excepción**.

⚠ La forma peligrosa

```
lock.acquire()  
contador += 1  
# lock.release() ← ¡si lo olvidas...!
```

El hilo se queda el cerrojo puesto **para siempre: deadlock**.



Regla de oro: el Lock protege la **sección crítica**, no el hilo. Todo el código que toque la variable compartida debe ir dentro del **mismo lock**.

Mini-chequeo: Lock



Responde sin mirar:

1. ¿Qué garantiza un Lock exactamente?
2. ¿Qué pasa si dos hilos llaman a `acquire()` a la vez?
3. ¿Por qué `with lock:` es mejor que `acquire()` / `release()` manuales?



Respuestas:

- Que solo **un hilo** entre en la sección crítica a la vez: exclusión mutua
- El primero entra; el segundo se **bloquea** hasta que el primero hace `release()`
- El `with` libera el lock solo al salir, **aunque haya excepción**; a mano es fácil olvidarlo y provocar un deadlock

RLock

La idea en una frase

“

Un Lock normal **no es reentrante**: si el mismo hilo intenta adquirirlo dos veces, se espera a sí mismo y se produce un **deadlock**. Un **RLock** (Reentrant Lock) sí permite que el mismo hilo lo adquiera varias veces.



El Lock no distingue entre «*otro hilo*» y «*yo mismo*»: la segunda `acquire()` espera a que se libere... un lock que **él mismo tiene puesto**.

El problema: Lock normal vs RLock

●●● reentrante.py

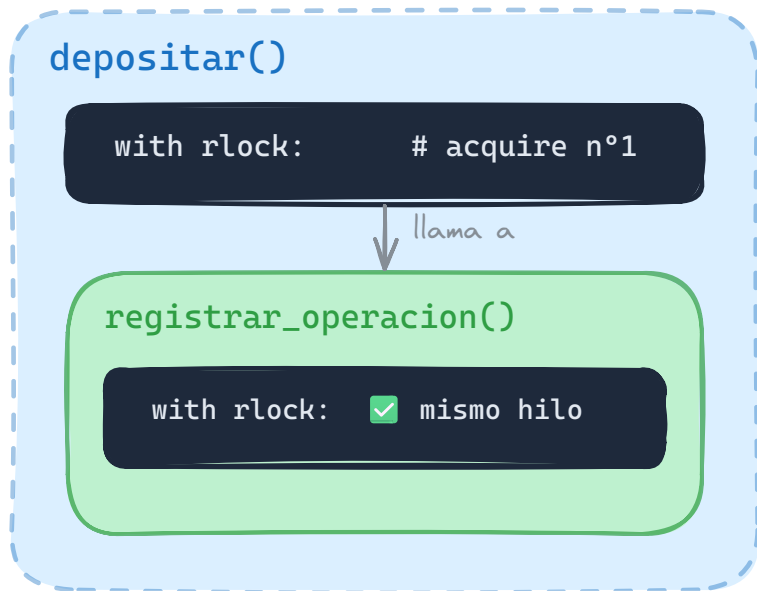
Python

```
import threading
# ❌ Lock normal -- DEADLOCK si el mismo hilo re-entra
lock = threading.Lock()
lock.acquire()
# lock.acquire() # ⚠️ el hilo se espera a sí mismo → 💀
# ✅ RLock -- el mismo hilo puede adquirirlo varias veces
rlock = threading.RLock()
rlock.acquire() # ok
rlock.acquire() # ok (mismo hilo)
rlock.release()
rlock.release() # liberado de verdad: el contador llega a 0
```

i El RLock lleva un **contador interno**: cada `acquire()` lo sube y cada `release()` lo baja. El lock solo se libera de verdad cuando el contador llega a 0.

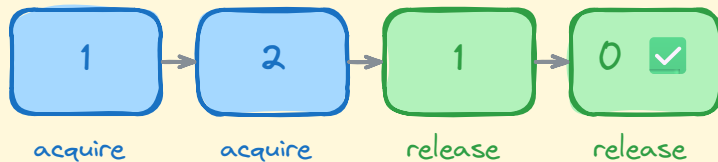
El caso típico: funciones que se llaman entre sí

RLock — el mismo hilo puede re-entrar



💡 También salva con recursión: la función se llama a sí misma y re-entra

contador interno del RLock



✗ Con un Lock normal, la 2ª `acquire()` espera a sí misma → 💀 deadlock

para el resto de casos: Lock por defecto

Lock vs RLock

Aspecto	Lock	RLock
¿Otro hilo puede adquirirlo?	Sí, cuando se libera	Sí, cuando se libera
¿El mismo hilo, dos veces?	❌ Deadlock	✅ Sí (contador interno)
¿Cuándo se libera de verdad?	En el primer <code>release()</code>	Cuando los <code>release()</code> igualan a los <code>acquire()</code>
Uso típico	Sección crítica simple	Funciones que se llaman entre sí / recursión
Coste	Mínimo	Mínimo (algo más de contabilidad)

⚠️ **Regla: usa Lock por defecto.** Cambia a RLock solo cuando de verdad el mismo hilo necesite re-entrar. Un RLock **no** evita que dos hilos distintos se pisen: para eso valen las mismas reglas del Lock.

Mini-chequeo: RLock



Responde en 30 segundos:

1. ¿Qué le pasa a un Lock normal si el mismo hilo lo adquiere dos veces?
2. ¿Cómo evita el RLock ese problema?
3. ¿Cuándo cambiarías de Lock a RLock?



Respuestas:

- **Deadlock:** se espera a sí mismo, porque la 2ª `acquire()` espera un lock que él tiene puesto
- Con un **contador interno:** sube con cada `acquire()`, baja con cada `release()`; libera al llegar a 0
- Cuando una función con el lock llama a **otra función que también lo quiere** (o recursión)

Resumen del bloque 01



Condición de carrera

± 1 son 3 pasos; el planificador corta entre ellos



Lock

Exclusión mutua; siempre

`with lock:`



RLock

Reentrante por contador interno; para funciones encadenadas



Bloque 02: ampliar el turno con **Semaphore**, coordinar **fases** con Barrier, avisar entre hilos con **Condition** y el patrón **productor-consumidor**.

